

Robust Frame-Level Detection for Deepfake Videos With Lightweight Bayesian Inference Weighting

Linjiang Zhou¹, Graduate Student Member, IEEE, Chao Ma, Member, IEEE, Zepeng Wang², Student Member, IEEE, Yixuan Zhang, Student Member, IEEE, Xiaochuan Shi³, Member, IEEE, and Libing Wu⁴, Member, IEEE

Abstract—Deepfake threatens the authenticity of the information in artificial intelligence Internet of Things (IoT) systems. Recently, several deepfake detection methods have been proposed in academia and industry for securing the authenticity of visual information in the face of artificial intelligence advances. Frame-level detection methods, a widely employed security method against deepfake, have a small model size and offer real-time responsiveness, despite basing their classification decision only on the information contained within the frame they are evaluating. We propose a new lightweight frame-level detection technique based on Bayesian inference weighting (BIW) to improve the robustness of existing frame-level detection models. Our proposed BIW technique employs the Naive Bayesian algorithm to estimate the reliability of any candidate model's detection results. Comprehensive experiments were conducted on the attacked data sets by four designed video interference approaches and edge computing platform, showing that BIW enhances the robustness of all the baselines and improves their detection accuracy with a real-time response.

Index Terms—Bayesian inference, deepfake video, frame-level detection, Internet of Things (IoT) security.

I. INTRODUCTION

NETWORK attacks based on artificial intelligence have been persistently occurring in recent years. Everyday millions of data are transmitted and processed through the Internet of Things (IoT) devices, and among these data, it contains a large amount of fake news, fake images, and fake videos [1]. Deepfake is an AI technology used to create face-swapping videos.

With the popularity of IoT and artificial intelligence technologies, deepfake technology can be easily used on some simple mobile devices to generate face-swapping videos. This

allows attackers to use deepfake technology to generate a vast number of fake videos for violent attacks or to deceive biometric recognition models [2]. Furthermore, the proliferation of fake videos on the Internet poses a significant threat to the content security of social media platforms [3]. For instance, attackers can use deepfake technology to deceive face recognition devices in IoT devices, even when these devices are equipped with live motion detection [4]. And in social networking platforms, users often utilize simple mobile devices to generate a large number of fake videos, for example, using mobile applications such as Zao.¹ While video websites and data centers can employ substantial GPU and storage resources for deepfake detection, some detection still needs to be performed on resource-constrained edge devices. Utilizing the computing resources available on users' mobile devices for local detection can mitigate the heavy consumption of network and data center resources while safeguarding user privacy. In both scenarios mentioned above, deepfake detection is required to do streaming and real-time detection, and thus is sensitive to resource consumption. Consequently, the deepfake threat in the IoT has garnered increasing attention in recent years, and researchers have focused on developing robust and efficient methods to detect and mitigate such attacks.

In recent years, to counter the threat of deepfake, many deepfake detection techniques have been widely used in mobile terminals and edge computing devices. Social media platforms, such as Twitter [5], Meta [6], Tiktok [7], etc. detect videos uploaded by users, prohibit the spread of deepfake fake videos, or label these videos with false information warnings. Deepfake detection techniques usually adopt deep learning models to detect suspicious videos or images. Deepfake detection techniques aim to automatically recognize deepfake videos by extracting detailed features, such as facial expressions, actions, and skin colors, or by combining them with the internal analysis of the deep learning model. In general, deepfake video detection techniques could be sorted into two categories: 1) frame-level method and 2) video level method.

A simple and intuitive method for detecting deepfake videos is to segment the video into separate frames and then detect whether there are any deepfake traces in each frame. Deep learning models are mostly employed in such

Manuscript received 5 May 2023; revised 4 October 2023 and 5 November 2023; accepted 23 November 2023. Date of publication 28 November 2023; date of current version 26 March 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFB3101100; in part by the National Natural Science Foundation of China, General Program under Grant 62272352; and in part by the Humanities and Social Sciences of Ministry of Education Planning Fund under Grant 21YJAZH073. (Linjiang Zhou and Chao Ma contributed equally to this work.) (Corresponding author: Xiaochuan Shi.)

Linjiang Zhou, Chao Ma, Zepeng Wang, Xiaochuan Shi, and Libing Wu was with the School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, Hubei, China (e-mail: linjiang@whu.edu.cn; chaoma@whu.edu.cn; wangzepeng@whu.edu.cn; shixiaochuan@whu.edu.cn; wu@whu.edu.cn).

Yixuan Zhang is with the School of Communication and Information Engineering, Chongqing University of Posts and Telecommunications, Chongqing 400065, China (e-mail: blsnedfaith@gmail.com).

Digital Object Identifier 10.1109/JIOT.2023.3337128

¹A social app developed by immomo.com in 2019 based on artificial intelligence and deepfake technology.

frame-level visual target detection methods. Most deep learning detection techniques are based on convolutional neural networks (CNNs), like MesoNet [8], Xception [9], F³-Net [10], EfficientNet [11], etc. Attention modules have recently been introduced to deepfake detection, like attentional and multiattentional detection models.

Deepfake algorithms usually generate fake videos frame by frame, which may produce unreasonable characteristics among sequential frames, such as a obviously abnormal blink frequency, uncoordinated changes in facial movements, or unnatural changes in facial brightness. In addition, imitating subtle changes, such as human heartbeat or breathing [12], is a nontrivial task for deepfake techniques. Therefore, video-level detection methods can usually achieve better detection accuracy by mining the aforementioned sequential characteristics among frames. For instance, a deepfake video detection method based on the blink frequency was proposed in [13].

Compared with video-level detection, frame-level detection has the characteristics of small model size and easily distributed real-time deployment. However, due to frame-by-frame detection, frame-level detection techniques tend to average the predicted results of all video frames and then evaluate the video's authenticity according to the average value. On one hand, the vanilla frame-level detection model shows significant performance degradation when analyzing video-level interference. On the other hand, although some learning-based weighted methods have been proposed to enhance detection robustness, these methods greatly increase the computational resources requirement, making them almost impossible to use on mobile terminals or edge devices. Therefore, this article proposes a new lightweight aggregation strategy based on Bayesian inference weighting (BIW) to improve the robustness of the frame-level detection model. Our contributions in this article can be summarized as follows.

- 1) By applying Bayes' theorem, a model-independent method named BIW is proposed to enhance the robustness of existing frame-level deepfake video detection models with minimal computational cost.
- 2) Three deepfake video data sets consisting of synthesized data with four kinds of common video interference in industry are designed and collected to comprehensively measure the robustness of frame-level detection models.
- 3) Comprehensive experiments are conducted on three real-world deepfake video data sets and show that our proposed BIW boosts the robustness of all baselines and improves the detection performance.
- 4) Efficiency experiments on edge computing devices have also demonstrated that our method requires only minimal computational resources and has the potential for widespread deployment on IoT computing terminals.

The remainder of this article is organized as follows. In Section II we introduced representative deepfake video detection methods proposed in recent years. Then, the proposed lightweight BIW for frame-level detection is illustrated in detail in Section III. In Section IV, experimental results are discussed to validate the effectiveness and efficiency of

our proposed BIW for enhancing the robustness of existing frame-level detection models with experimental settings provided. Conclusions are drawn in Section V to summarize our work.

II. RELATED WORK

Deepfake production and detection techniques, respectively, correspond to offense and defense. In recent years, as deepfake techniques have rapidly evolved from auto-encoders to generative adversarial networks (GANs), so too have the corresponding deepfake detection techniques, which went from manual forensics [14] to deep learning detection methods. Deepfake videos usually choose to exchange faces in the central area of the face instead of tampering with the whole face. Visual differences appear between the forged area in the central part of the face and the real area at the edge of the face, in terms of brightness, color and pixelation. These differences can be distinguished by machine learning algorithms [15], deep learning models (such as CNN) [16], or other classification algorithms. There are usually two data-driven technical routes for deepfake video detection – video-level detection and frame-level detection.

A. Deepfake Video Generation

In recent years, there has been a significant increase in the development of deepfake generation techniques based on GANs [17]. Some of these techniques have even been packaged into easy-to-use software or scripts, enabling users to easily generate deepfake videos on simple computing devices. Two widely adopted deepfake video generation tools will be introduced.

FaceSwap (FS) [18] represents typical work in the field of facial feature extraction and fusion. Its architecture includes an encoder, a decoder, and a discriminator, which work together to extract facial features, restore individual features through deconvolution, and fuse features via a residual network. Additionally, the adversarial network is employed to improve the performance of both the encoder and decoder. FS also features a user-friendly graphics interface that makes it accessible for novices to use; however, it is important to note that configuration errors may result in difficult-to-resolve issues.

DeepFaceLab (DFL) [19] is an integrated open-source system with a clean-state pipeline design, that achieves photorealistic face-swapping results without painful tuning. DeepFaceLab consists of an encoder, inter, and decoders. The task of the encoder is to acquire the features within the human face, including character expression, facial feature position, etc. Then the inter extracts these features and transmits them to the decoder, which restores the features to the original face. The GAN structure, which is widely applied to numerous real-world applications [20], [21], enables DFL with an incredible learning and imitation capability. DFL is less dependent on the GPU and is easily deployable and trainable, which makes it widely used. At the same time, it can also manually intercept the face.

B. Video-Level Detection

Deepfake videos are usually synthesized frame by frame, which might generate unrealistic phenomena between frames, such as significantly blink frequency lower than usual, uncoordinated changes in facial movements, and unnatural changes in facial brightness [13].

Authentic optical flow can be reflected in the optical flow field by considering the difference between computer-synthesized fake videos and camera-taken real videos. I3D [22] was proposed to learn spatio-temporal features with 3-D ConvNet, and [19] used the LK algorithm and PWC-Net to identify forged video frames as optical streams. Temporal inconsistency between real and fake videos also could be leveraged for detecting fake videos. For instance, STIL [23] designed a spatial-temporal inconsistency learning block to extract the fine-grained temporal features of videos for classifying videos. Similarly, Zhao et al. [24] proposed the self-consistency loss to learn the deep consistency features of potentially forged videos. Also, temporal learning nodes [25] were adopted for simultaneously learning frame-level and video-level features to make the detection more robust.

The video-level analysis of video-level detection models cost numerous computational resources. Video-level detection models were usually announced to achieve higher accuracy, but in industrial applications, these models require much higher performance CPUs and GPUs than frame-level detection models. Therefore, few video-level detection models were employed in visual perception system of industrial applications.

C. Frame-Level Detection

The basic idea of frame-level detection is to divide the video into a series of consecutive frames, analyze each frame separately, and then estimate the real/fake probability of the entire video. Most frame-level detection models are based on CNN-like structures, but in recent years a novel vision transformer technique has been applied to deepfake video detection. Moreover, some weighting or aggregation methods were developed to enhance existing models. However, these weighting or aggregation methods are not specifically designed to make a balance between the model's computational cost and its robustness.

With a few layers that focus on the mesoscopic properties of images, MesoNet [8] can automatically and efficiently detect face tampering in videos. MesoNet consists of two models, Meso-4 and MesoInception-4. Both of them employ fewer parameters and layers and are easy to train. Rössler et al. [9] developed XceptionNet to detect fake videos, which has shown notable performance over several data sets. EfficientNet [11], another efficient deep learning model for image classification, is one of the feasible alternatives for deepfake video detection.

The convolutional vision transformer was recently introduced to identify fake videos [26], where its attention modules showed the competitive capability of extracting fine-grained features from target videos. Furthermore, Zhao et al. [27] applied multiple attentions and texture learning blocks to

achieve higher deepfake detection accuracy. Deep convolutional pooling transformer [28] also were introduced to improve detection performance.

A couple of weighting approaches modified the detection strategy to enhance the performance and robustness of frame-level detection models. DeepfakeStack [29] integrates a series of state-of-the-art detection models and uses a stacking ensemble method to determine the detection results. AFW [30] designed a face weighting layer to learn the temporal features after the extraction of frame-level detection models. S-MIT [31] is a weighting approach to detecting potentially faked frames based on multiple instance learning, which could boost the robustness of the frame-level detection model on partially attacked data sets.

For the aforementioned frame-level detection models, although they have employed diverse neural networks, almost all of them average the predicted output results of all frames to make the decision whether the video is real or fake. However, such kind of averaging strategy does not work well in the case that the predicted results are inconsistent across frames, which makes the detection models lack robustness. As a consequence, the misjudgment of a few frames will result in the wrong decision of the detection model. Furthermore, video-level detection models requires a large amount GPUs computational resources, which makes them not suitable for edge computing devices with low performance. Therefore, in this article, we aim to propose a new lightweight results aggregation strategy for existing frame-level detection models to further improve their robustness while retaining their accuracy.

III. METHODOLOGY

In this section, an efficient approach named BIW is proposed to enhance the robustness of existing frame-level deepfake video models. To help the audience understand our proposed BIW, Bayes' theorem is reviewed in Section III-A. And then the BIW is illustrated in detail in Section III-B. In Section III-C, the implications of our proposed BIW are discussed.

A. Bayes' Theorem

One of the most important applications of Bayes' theorem is its capability of inferring the posterior probability from prior probabilities and established facts. If $\{A_1, A_2, \dots, A_n\}$ is a set of disjoint events, i.e., $P(A_i \cap A_j) = 0, i \neq j$ and $P(A_1 \cup A_2 \cup \dots \cup A_n) = 1$. Therefore, the probability $P(B)$ of event B could be defined as shown in

$$P(B) = \sum_{i=1}^n P(B|A_i)P(A_i). \quad (1)$$

Thus, for a given event B and event A_k , the conditional probability can be expressed as shown in

$$P(A_k|B) = \frac{P(B|A_k)P(A_k)}{P(B)} = \frac{P(B|A_k)P(A_k)}{\sum_{i=1}^n P(B|A_i)P(A_i)}. \quad (2)$$

B. Bayesian Inference Weighting

The procedure of BIW contains two steps: 1) segment reliability inference and 2) weighted aggregation. Fig. 1 shows

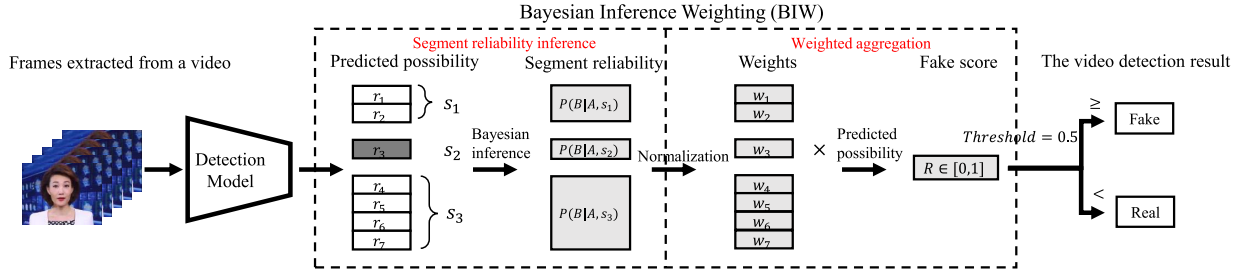


Fig. 1. Overview of BIW. BIW generates the fake score of the video by processing the predicted probability of the frame-level detection model for each frame.

TABLE I
HYPOTHETICAL CONFUSION MATRIX

	<i>fake</i>	<i>real</i>
<i>fake</i>	a	b
<i>real</i>	c	d

the overview of BIW. In the first step, frame prediction possibility from detection models will be separated into several segments according to their possibility values, and BIW infers the reliability of each segment with Bayesian inference. In the next step, the normalization approach is used to transform the segment reliability values into the weights, and the fake score is the weighted sum of the frame predicted possibility with these weights. In general, if the video's fake score is greater than or equal to the threshold (set to 0.5), the video is detected as a fake video, otherwise a real video.

Segment Reliability Inference: For a video V , it could be assumed that the fake score of model M 's detection is $R_n = \{r_1, r_2, \dots, r_n\}$, where n is the number of extracted video frames, and the weight corresponding to result r_i is w_i . When a series of consecutive video frames are detected as being of the same category (real/fake), we assume that the reliability of this subsequence of frames should be higher than that of a single frame being categorized as opposite of its surroundings. Thus, the target problem could be briefly described: if model M has a certain probability of misjudgment, and when the model judges that the results of k consecutive video frames are consistent, that is, when there is a sequence $s = \{r_{j+1}, r_{j+2}, \dots, r_{j+l}\}$ containing l results in which $r_{j+i} \geq 0.5 \forall r_{j+i} \in s$ or $r_{j+i} < 0.5 \forall r_{j+i} \in s$, how can we ensure that the judgment of the model M is correct?

We can assume a hypothetical confusion matrix of training or validation, as shown in Table I. *fake* and *real* represent the samples with ground truth labels as either fake or real videos, while $\hat{fake}$ and $\hat{real}$ represent the samples with fake and real prediction results. And a, b, c , and d are the corresponding the number of video samples labeled according to their ground truth and estimated category.

For convenience, consider a case where $r_{j+i} < 0.5 \forall r_{j+i} \in s$. The event of the occurrence of the sequence s is recorded as $\hat{fake}_s$ and the event of video V as being fake is recorded as $fake_s$. Then the probability that the result of this sequence s is right is recorded as $P(fake_s | \hat{fake}_s)$, and (3) can be obtained according to

$$P(fake_s | \hat{fake}_s) = \frac{P(fake_s)P(\hat{fake}_s | fake_s)}{P(\hat{fake}_s)}. \quad (3)$$

Next, we consider the items in (3) in turn. $P(\hat{fake}_s)$ is the probability of judging that the segment s is fake, $P(fake_s)$ is the ground truth probability that the segment s is fake, and $P(fake_s | \hat{fake}_s)$ is the probability of judging that the segment s is fake after knowing the fact that the segment s is fake. To estimate the value of each item in (3), we use the confusion matrix of training or validation to calculate each item according to the features of machine learning approaches.

The definition of $P(fake_s)$ is the ground truth possibility that the segment s is fake. To more accurately estimate $P(fake_s)$, we define it as the difference between the number of real and fake frames, and consider the segment as fake when this difference is greater than or equal to zero. Unfortunately, it is quite complex to calculate the values of $P(fake_s)$ and $P(fake_s | \hat{fake}_s)$.

For simplicity, we could consider the case of $l = 1$ first, where l denotes the length of each segment. In this case, it is easy to compute the value of $P(fake_s | \hat{fake}_s)$, as shown in

$$P(fake_s | \hat{fake}_s) = \frac{a}{a+b}. \quad (4)$$

Since the detection result between frames is an independent event, when $l > 1$, each frame can be regarded as the detection result when $l = 1$. So we can sum up the possibility of all combinations to obtain the value of $P(fake_s | \hat{fake}_s)$ when $l > 1$. Thus, $P(fake_s | \hat{fake}_s)$ can be calculated as shown in

$$P(fake_s | \hat{fake}_s) = \sum_{i=\lceil \frac{k}{2} \rceil}^l \binom{l}{i} \left(\frac{a}{a+b}\right)^i \left(\frac{b}{a+b}\right)^{l-i}. \quad (5)$$

Weighted Aggregation: In the cases when $r_{j+i} \geq 0.5 \forall r_{j+i} \in s$, the calculation process is the same. The detection result of video V can be divided into such a set $S = \{s_1, s_2, \dots, s_N\}$ of continuous sequences s_k , where N is the number of sequences s_k . Given the segment length l_k , the value of reliability inference p_k and the segment weight w_i^s correspond to each segment s_k , the segment weight w_i^s is as shown in

$$w_i^s = \frac{a_i}{\sum_{i=1}^n a_i}, a_i = \frac{p_i}{\bar{p}} \times l_i. \quad (6)$$

The final result R of this process is shown in

$$R = \sum_{i=1}^n w_i^s \bar{r}_i, \bar{r}_i = \frac{\sum_{j=1}^k r_j}{k}, r_j \in s_i. \quad (7)$$

Finally, our proposed BIW helps the frame-level detection model make the decision according to the fake score R . In general, the fake score R is the basis for deepfake video detection.

C. Implications

Intrinsically, BIW is a result aggregator rather than a model combiner. Because BIW only depends on the numerical values in the confusion matrix of the model, it is a model-independent enhancement method, which can be applied to any model that conforms to the frame-level deepfake video detection process. Also, BIW will not damage the performance of the model under normal detection settings, but indeed enhances the performance of the model under interference. BIW does not require extra training, learning, and other data-driven processes, and will not introduce additional blocks or modules into existing models and thus brings negligible computation burden.

Bayesian inference is used to calculate the weights in BIW, which differs from simply using averaging or other weighting strategies. Since BIW can guarantee that every decision of the model is almost equally trusted, BIW will not degrade the accuracy of the model. In (5), if the detection accuracy is close to 100%, BIW will degenerate into an average strategy. When the model accuracy is relatively low, the aggregation result will rely more on continuous and consistent prediction results, which could improve the robustness of the model by correcting erroneous predictions. The frame inference also was used in FInfer [32], but FInfer adopted a learning-based process to obtain the frame feature in high-visual-quality video. While they must sample the feature in video-level, which leads to a multiple-fold increase in computational complexity.

The weighting value is directly calculated by the numerical values in the confusion matrix and the length of continuous and consistent prediction results. It is different from other weighting approaches, like face weighting [30] and multiple instance learning [31], which are based on data-driven methods. BIW does not require making any modification to the structure or training process of the original detection model, nor does it need to extract any parameter of the original detection model. It can even be applied to a black box detection model.

BIW has almost negligible time complexity, as the calculation of (5) only depends on segment length k . The complexity of calculating the combination number $\binom{l}{k}$ is only $O(k^2)$, which is much less than the complexity of the frame-level detection model itself. Considering the application, for a known detection model, we could even calculate all possible values of $P(B | A)$ in advance. BIW can retain the high efficiency of the frame-level detection method and adapt to application scenarios with high-response speed.

TABLE II
IIE-FACE USED VIDEO DATA OVERVIEW

Type	Video Number	Fake-Real Ratio	Enhanced
Train	8000	6:4	False
Validation	2000	6:4	False

As an algorithm designed to defend against deepfake attacks, BIW actually carries certain security risks. The attackers can potentially attack our algorithm by obtaining the prestorage weight table in advance and manipulating the frame sequence accordingly. Therefore, in practical applications, we recommend appropriately safeguarding the weight table to prevent exploitation and tampering by attackers. For instance, asymmetric key encryption techniques can be employed to encrypt the weight table in BIW using a public key, ensuring that the risk information of BIW can only be accessed in trusted devices.

Involving legal risks, such as privacy, we recommend adopting a strategy of local computation. The information collected on the user's device should be processed locally without being uploaded to the server. Since its low-resource usage and fast computation speed, BIW can be deployed for local computation on the user's device. Therefore, in deepfake detection applications that involve privacy concerns, our method is more suitable for privacy-preserving computation compared to other detection models that rely on high-computing power servers.

IV. EVALUATION

Two kinds of experiments were employed to test the accuracy and robustness of the benchmark models compared to the BIW enhanced models. Section IV-A introduces three real-world deepfake data sets: 1) IIE-FACE; 2) FaceForensics++ [9]; and 3) DFDC [33] and their interfered versions. A robustness criteria that is designed for quantitative comparison between models is introduced in Section IV-C. The results generated from the two main indicators, accuracy and robustness, are given in Section IV-D.²

A. Data Sets

IIE-FACE was launched in the Fake Media Forensic Challenge of the Chinese Society of Image and Graphics (CSIG) [34]. It includes face videos collected from young Chinese people speaking in different scenes. Four methods are used for generating deepfake videos, including DeepFaceLab [19], FS, FS-Gan [18], Recycle-Gan [35], and ALAE [36]. In our experiment, IIE-FACE was divided into two parts, the validation set, and the training set, according to the ratio of 8:2. The composition of used video data in IIE-FACE is shown in Table II.

Since IIE-FACE lacks label information for generating interference videos, we constructed a separate test set corresponding to IIE-FACE for testing the performance of the model and for the simulation of interference videos. These videos were selected from an industrial video processing field,

²We will open-source our code and data sets to promote scientific sharing and reproducibility.

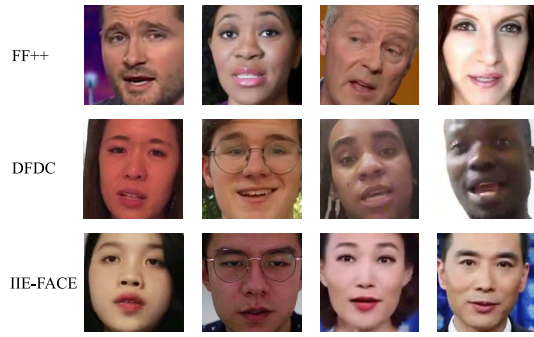


Fig. 2. Overview of different types of faces in FF++, DFDC, and IIE-FACE.

a TV program called ANCHOR SAID SIMULCAST of China Central Television (CCTV), which contains videos of nine hosts. The open-source project DeepFaceLab [19] was used to facilitate the production of deepfake videos.

FaceForensics++ (FF++) [9] is a collection of data sets containing a variety of deepfake videos, which are widely used to train and test deepfake detection models in IoT [37], blockchain [38], social media [39], etc. FF++ contains five subdata sets, which are DeepFakes (DF), Face2Face (F2F), FS, FaceShifter (FSF), and NeuralTextures (NT). The deepfake videos in these five subdata sets are generated using the corresponding deepfake generation techniques.

DFDC [40] is a deepfake video data set released by the DeepFake Detection Competition, which contains both preview and full versions. DFDC contains a large number of deepfake videos, but the full version lacks the corresponding label information for generating interference videos, so we only use the preview version for our experiments.

IIE-FACE, FF++, and DFDC contain a large variety of information for each face, such as skin tone, age, gender, and scene, which can measure the performance and generalization ability of the detection model more comprehensively. As shown in Fig. 2, these three data sets contain a rich sample of human faces. The FF++ data set comprises deepfake data sets generated using five advanced deepfake methods (DF, Face2Face, FS, FaceShifter, and NT). It was initially released in 2019 and has been continuously updated.³ The version we used corresponds to the latest version available. The DFDC data set combines eight different deepfake methods and was jointly released by Amazon, Facebook, Microsoft, and PAI.⁴ Additionally, we specifically created a custom deepfake data set using the widely used deepfake tool called DeepfaceLab.⁵ This data set incorporates the most common news videos on the Internet to swap faces. Therefore, by training and testing our detection method on these diverse data sets, we have demonstrated its effectiveness in countering state-of-the-art deepfake methods.

B. Video Interference

Video interference generally occurs in the video transmission or video processing operation. In [31] and [41],

³<https://github.com/ondyari/FaceForensics>

⁴<https://www.kaggle.com/c/deepfake-detection-challenge/>

⁵<https://github.com/iperov/DeepFaceLab>

Algorithm 1 Frame Loss Simulation Algorithm

```

1: FRAME-REPLACING(Source, Target, Rate)
2:   Indexes = Random(Source.Length, Rate)
3:   Mix = Copy(Source)
4:   FOR index IN Indexes
5:     Mix[index] = Target[index]
6:   ENDFOR
7:   RETURN Mix

```

Algorithm 2 Coding Error Simulation Algorithm

```

1: AREA-ERASING(Video, Rate)
2:   FOR Frame IN Video
3:     X = RandomInt(0, Rate * Video.Height)
4:     Y = RandomInt(0, Video.Width - (Video.Size / (Video.Height - X)))
5:     H = RandomInt(Video.Size / (Video.Width - Y) + 1, Video.Height - X)
6:     W = Video.Size / H
7:     Frame[X: X + H, Y: Y + W] = [0, 0, 0]
8:   ENDFOR
9:   RETURN Video

```

some common video interferences in industrial wireless and electronic processing units are mentioned, like frame loss, coding error, random noise, electromagnetic interference, etc. To simulate these possible video interferences, we designed four algorithms to easily generate different interference videos. These four algorithms correspond to four kinds of video interference: 1) frame loss; 2) coding errors; 3) random noise; and 4) electromagnetic interference.

Frame loss is usually caused by video cutting, splicing, etc. In order to simulate the process, real videos and fake videos are processed for corresponding real-fake pairs. As shown in Algorithm 1, in the fake video of a real-fake pair, some randomly selected frames according to a specific replacement proportion are replaced with the corresponding frames in the real video of the same real-fake pair. The interference rate should be capped under 50%.

Coding errors often occur when a video is loading, encoding, and decoding. Some areas of the frames in the video are missing due to encoding errors. So we designed Algorithm 2 to randomly erase some areas in the video to simulate this interference.

Random noise is very common in video interference and is often used to test the robustness of video processing methods. There are many kinds of random noise, such as Gaussian noise, Poisson noise, etc. As shown in Algorithm 3, we use 0/1 noise to simulate random noise.

Electromagnetic interference is a relatively common type of interference in the process of wireless video transmission. Just like the interference that appears in wireless satellite TV, electromagnetic interference usually makes the video affected by streak-like noise. Algorithm 4 simulates the electromagnetic interference.

These four interference algorithms will be used on the test sets of IIE-FACE, FF++, and DFDC to obtain the

Algorithm 3 Random Noise Simulation Algorithm

```

1: NOISE-ADDING(Video, Rate)
2:   FOR Frame IN Video
3:     Mask = RandomMask(Video.Size, Rate)
4:     MaskZero = RandomChoice(Mask, 0.5)
5:     MaskOne = Mask - MaskZero
6:     Frame[MaskZero] = [0, 0, 0]
7:     Frame[MaskOne] = [255, 255, 255]
8:   ENDFOR
9:   RETURN Video

```

Algorithm 4 Electromagnetic Interference Simulation Algorithm

```

1: STREAK-ADDING(Video, Rate)
2:   FOR Frame IN Video
3:     StreakHeight = Rate * Video.Height)
4:     Index = Random(Video.Height, StreakHeight)
5:     Frame[Index, :] = [0, 0, 0]
6:   ENDFOR
7:   RETURN Video

```

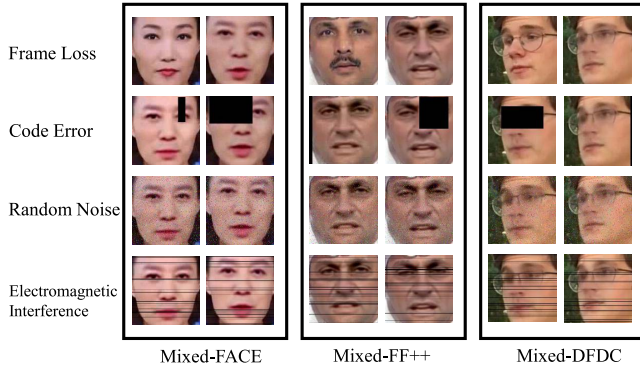


Fig. 3. Examples of faces in interference data sets: Mixed-FACE, Mixed-FF++, and Mixed-DFDC.

TABLE III
INTERFERENCE RATE SETTINGS

Interference	Interference Rate				
Flame Loss	0.10	0.20	0.30	0.40	0.50
Code Error	0.04	0.08	0.12	0.16	0.20
Random Noise	0.02	0.04	0.06	0.08	0.10
Electromagnetic Interference	0.04	0.08	0.12	0.16	0.20

corresponding interference data sets named Mixed-FACE, Mixed-FF++, and Mixed-DFDC. Fig. 3 shows some of the face data in Mixed-FACE, Mixed-FF++, and Mixed-DFDC. For each interference algorithm, we set five interference rates separately to measure the robustness of the model comprehensively. Table III shows the interference rate settings.

C. Robustness Evaluation Criterion

In the case of normal model detection, it is difficult to compare the robustness of the different models. The frame prediction value, which is the possibility that the single frame is real or fake, will be considered as the classification basis during the predicting process of the frame-level detection

TABLE IV
ACCURACY OF DIFFERENT MODELS ON IIE-FACE, FF++ AND DFDC.
FF++ INCLUDES FIVE SUBDATA SETS: DF, F2F, FS, FST, NT.
ACCURACY DIFFERENCES ARE MARKED IN BOLD

Datasets		IIE-FACE	FF++					DFDC
Models			DF	F2F	FS	FST	NT	
Meso4		0.9078	0.9964	0.9893	0.9000	0.9929	0.8214	0.8549
Meso4 BIW		0.9078	0.9964	0.9893	0.9035	0.9929	0.8286	0.8549
MesoInception4		0.8947	0.9893	0.9929	0.9714	0.9179	0.9857	0.8225
MesoInception4 BIW		0.8947	0.9893	0.9929	0.9714	0.9143	0.9857	0.8225
XceptionNet		1.0000	0.9929	0.9893	0.9964	0.9964	0.9857	0.8515
XceptionNet BIW		1.0000	0.9964	0.9893	0.9964	0.9964	0.9857	0.8515
EfficientNet-B4		0.9013	0.9964	0.9821	0.9714	0.9929	0.9107	0.8430
EfficientNet-B4 BIW		0.9013	0.9964	0.9821	0.9750	0.9929	0.9142	0.8430

models. By adopting a certain strategy, usually averaging, the frame-level prediction value is integrated into the video-level prediction fake score. However, averaging may lead to misjudgment, especially when the predicted fake score is close to 0.5. For this reason, models with fake scores close to 0.5 can be considered less reliable. Therefore, we define the system robustness value R in

$$R = \frac{\sum_{i=1}^N |P_i - 0.5|}{N}. \quad (8)$$

In (8), N is the total number of video samples in the robustness test. P_i represents the comprehensive prediction value of the i th video sample. By definition, R is a number calculated by the average distance between each frame prediction value and 0.5, which puts it between 0 and 0.5. The higher the robustness value R , the more reliable and robust the model is. The robustness value R can help to further quantitatively evaluate the potential robustness of the model.

D. Result Discussion

Meso-4, MesoInception-4, XceptionNet, and EfficientNet-B4 have proven to be efficient in frame-level deepfake video detection. Considering both efficiency and performance, the four models were adopted as benchmarks. The hyperparameters of Meso-4 [8], MesoInception-4 [8], XceptionNet [9], and EfficientNet-B4 [11] are set to their default values as defined in their standard models. The accuracy and robustness of these models will be compared with their BIW enhanced variations, respectively. We further simulated the application scenario of video detection in IoT devices and conducted efficiency testing on the BIW algorithm.

1) *Evaluation of Model Accuracy*: The benchmark models as well as their BIW enhanced versions were trained and tested on IIE-FACE, FF++, and DFDC. The parameters of BIW are set according to the confusion matrix of validation results. Table IV shows that the models with BIW get almost the same accuracy as the originals and even higher on a few data sets, which means that BIW can slightly improve the performance of the model even when the model is detected normally.

Next, the baseline models and their BIW-enhanced version were tested for accuracy on the interference data set, Mixed-FACE, Mixed-FF++, and Mixed-DFDC. As shown in Table IV, the accuracy of the benchmark model is high on all three data sets. As mentioned in Section III-C, BIW degrades to an average algorithm when the model accuracy approaches 100%. Therefore, we set to add a penalty factor α to prevent the degradation of BIW, because the accuracy of the model is



Prediction	0.6199	0.0000	1.0000	1.0000	0.0207	0.9033	0.0000	0.0003	1.0000	1.0000	1.0000	0.0083	1.0000	0.0000	0.0000	Result
Average weight	0.0667	0.0667	0.0667	0.0667	0.0667	0.0667	0.0667	0.0667	0.0667	0.0667	0.0667	0.0667	0.0667	0.0667	0.0667	0.5035
BIW weight	0.0612	0.0688	0.0679	0.0679	0.0688	0.0612	0.0688	0.0688	0.0664	0.0664	0.0664	0.0688	0.0612	0.0688	0.0688	0.4914

Fig. 4. Weighting example of BIW weighting. These real faces were detected by Meso4 from FS in Mixed-FF++ with random noise interference. The interference rate of this example is 0.08, and the sample id in FF++ is 102.

TABLE V
AVERAGE ACCURACY OF DIFFERENT MODELS ON MIXED-FACE, MIXED-FF++ AND MIXED-DFDC. THE HIGHER ACCURACY VALUES ARE MARKED IN BOLD

Datasets		Mixed-FF++					Mixed-DFDC
Models	Mixed-FACE	DF	F2F	FS	FST	NT	
Meso4	0.5974	0.7889	0.7754	0.5173	0.6205	0.6523	0.6796
Meso4 BIW	0.6046	0.7941	0.7807	0.5204	0.6220	0.6532	0.6863
MesoInception4	0.6365	0.7052	0.6423	0.4891	0.6741	0.7243	0.5899
MesoInception4 BIW	0.6405	0.7129	0.6489	0.4896	0.6754	0.7264	0.5996
XceptionNet	0.7809	0.8680	0.8162	0.5996	0.7332	0.8166	0.5584
XceptionNet BIW	0.7720	0.8705	0.8186	0.6088	0.7384	0.8179	0.5678
EfficientNet-B4	0.7638	0.8609	0.7946	0.4907	0.7332	0.6650	0.5168
EfficientNet-B4 BIW	0.7553	0.8636	0.7979	0.4943	0.7357	0.6746	0.5293

TABLE VI
ROBUSTNESS VALUE OF DIFFERENT MODELS ON IIE-FACE, FF++, AND DFDC. THE HIGHER ROBUSTNESS VALUES ARE MARKED IN BOLD, AND THERE IS NO REFERENCE VALUE FOR THE ROBUSTNESS VALUE OF THE TERM FOR WHICH THE ACCURACY HAS CHANGED

Datasets		IIE-FACE		FF++				DFDC
Model		DF	F2F	FS	FST	NT		
Meso4		0.3955	0.4900	0.4866	\	0.4852	0.4083	
Meso4 BIW		0.3962	0.4900	0.4867	\	0.4853	0.4084	
MesoInception4		0.3817	0.4886	0.4809	0.4608	\	0.4192	
MesoInception4 BIW		0.3818	0.4886	0.4809	0.4609	\	0.4194	
XceptionNet		0.4531	\	0.4845	0.4767	0.4782	0.4029	
XceptionNet BIW		0.4532	\	0.4846	0.4767	0.4782	0.4820	
EfficientNet-B4		0.4011	0.4831	0.4790	\	0.4867	0.4159	
EfficientNet-B4 BIW		0.4011	0.4831	0.4791	\	0.4867	0.4159	

bound to drop a lot when the videos contain interference. The penalty factor α is used as shown in

$$b = \begin{cases} b, & \text{accuracy} < 1 - \alpha \\ \alpha \times d + b, & \text{accuracy} \geq 1 - \alpha. \end{cases} \quad (9)$$

Both b and d in (9) are the values in the confusion matrix in Table I. The reason why a penalty factor is added to b and d is because deepfake detection requires a higher precision rate, so we just add a penalty factor to b and d without a and c . As shown in Table V, in our interference experiments, the model accuracy is mostly around 0.6 after interference so we set $\alpha = 0.4$ by empirical experience.

As shown in Table V, we calculated the average accuracy of the model with different interference methods and interference rates in the three interference data sets. It can be clearly found that BIW improves the accuracy of the model for almost all four benchmark models in all interference data sets, on average by 0.39%. For models and data sets with low robustness, such as EfficientNet-B4 on the DFDC data set, BIW can improve the accuracy by 1.25%. Fig. 4 shows an example where BIW has corrected the error detection by weighting.

In our experiments, we utilized training and testing data sets that covered four different resolutions: 1) 640×360 ; 2) 640×480 ; 3) 1920×1080 ; and 4) 1440×2560 . The IIE-FACE and FF++ data sets had relatively lower resolutions, while the DFDC data set covered commonly encountered resolutions on the Internet, such as 1080p and 2k. The experimental results presented in Table V indicate that BIW can improve the performance of frame-level detection models across different resolution scenarios.

2) *Evaluation of Model Robustness*: When tested on the noninterference data set, there was little difference between the accuracy of the BIW and the average method, as shown in Table IV. Therefore, to further measure the enhancement of

BIW for robustness, we measured the term in Table IV where the accuracy of BIW and the average method are equal using the robustness value R in (8).

As shown in Table VI, BIW can improve the robustness values of most benchmark models. The robustness-enhancing effect of BIW is more obvious when the robustness of the model is low, such as in the four benchmark models tested in IIE-FACE. On average, BIW can improve the robustness value of the model by 0.10%.

In the BIW algorithm, the inferential properties of Bayesian inference enable the weighting process to minimize the occurrence of misjudgments. We employ a conservative weighting strategy in (7), which allows BIW to handle videos that are on the decision boundary more cautiously. Due to the large volume of experimental data, we have selected a representative segment to demonstrate the effect of BIW on false positive and true negative samples. Fig. 5 illustrates a comparison of the confusion matrix between Meso4 model and the BIW enhanced model on the mixed-FACE data set. It is evident that as the interference rate gradually increases, BIW continuously attempts to correct some of the model's predictions. This behavior is a result of the enhancement provided by BIW, which aims to mitigate the impact of interference and improve the model's performance.

3) *Evaluation of Model Efficiency*: To simulate realistic edge computing scenarios in IoT environments, we emulated the video detection process and conducted end-to-end detection on each video in the test data set. The NVIDIA Jetson platform is a prevalent computing platform for embedded IoT applications, widely utilized in IoT devices, such as autonomous driving, network video recorders, home robots, and smart gateways. Therefore, we selected the NVIDIA

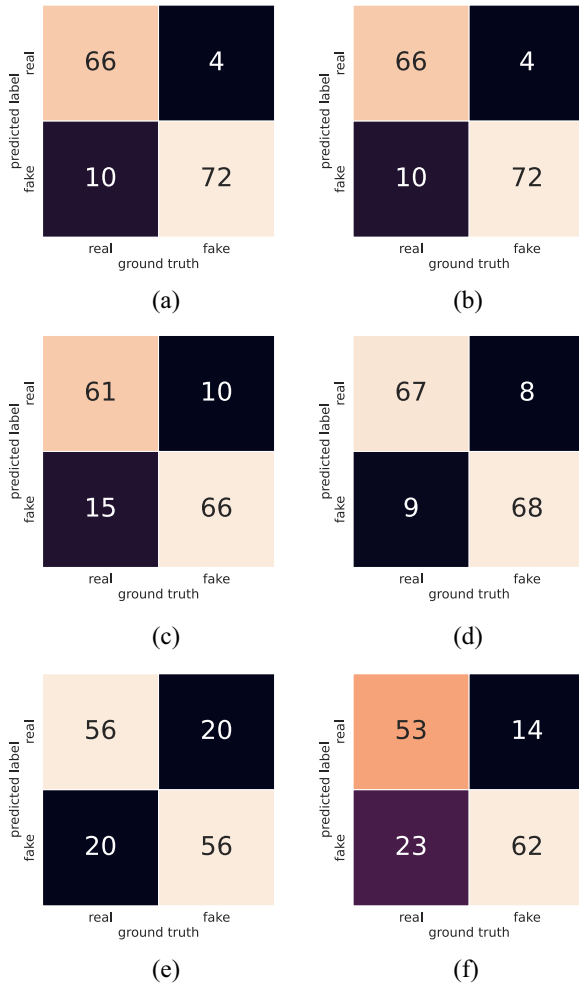


Fig. 5. Confusion matrix for Meso4, with enhancement (BIW) and without enhancement (RAW), on Mixed-FACE using different interference rates with Frame Loss. (a) RAW (0). (b) BIW (0). (c) RAW (0.2). (d) BIW (0.2). (e) RAW (0.4). (f) BIW (0.4).

TABLE VII
HARDWARE PARAMETERS OF THE SIMULATION PLATFORM

Component	Model
GPU	128-core NVIDIA Maxwell
CPU	Quad-core ARM A57 @ 1.43 GHz
Memory	4GB 64-bit LPDDR4 25.6GB/s
POWER	5W
AI Perf [42]	472 GFLOPS

Jetson Nano module as the simulation platform. The computational parameters of the NVIDIA Jetson Nano platform are provided in Table VII.

In terms of model deployment, we first conducted pre-training on all models on a server computing platform. Subsequently, we deployed all detection models and the BIW algorithm based on the Jetpack SDK provided by the Jetson platform. The computing libraries and software utilized in this process are listed in Table VIII. The baselines with BIW enhanced version were tested with the three data sets on the IoT platform to evaluate their detection computational cost. All baselines were built with ONNX Runtime and BIW was built with Numpy. We iterated the simulation 10 times to comprehensively measure the model's time cost.

TABLE VIII
COMPUTING LIBRARIES AND SOFTWARE INFORMATION

Name	Version
Jetpack	4.6.3
Ubuntu	18.04
CUDA	10.2.300
cuDNN	8.2.1.32
TensorRT	8.2
Python	3.6.9
ONNX Runtime	1.10.0

We designed experiments to simulate real-time video detection, which involved simulating the I/O mode of streaming video processing and conducting experiments on embedded devices. The IIE-FACE data set used in our experiments was collected from everyday encounters with online videos, and we also utilized the widely used large-scale face data sets FF++ and DFDC. These experiments simulate scenarios involving the processing and analysis of a large amount of image data in everyday life. In terms of processing time, our lightest model achieved an average detection speed of 5.76 ms per 1-s video. Table IX provides an overview of our complete experimental results. In practical applications, these delays are negligible for users.

Regarding the resource utilization, the BIW module introduces minimal additional computational time. Fig. 6 illustrates the ratio of the BIW module's runtime. It shows that the BIW module's computational overhead is negligible, indicating that it does not significantly impact the overall processing time.

We compared the model parameter sizes and detection accuracy of these video-level detection models and frame-level detection models, all evaluated on the FF++ data set. It is important to note that these video-level detection models, due to their large memory footprint and resource consumption, cannot be run on embedded experimental devices. Fig. 7 provides a visual representation of the relationship between the performance and resource consumption of these models. BIW requires minimal computational resources, whose consumption is almost negligible compared to the frame-level detection model or other video-level robust enhanced method.

Overall, through experiments on real-world data sets and IoT platforms, BIW can help the model directly improve its accuracy in the face of disturbances, thus achieving enhanced robustness. Real-time detection and low-resource consumption are critical requirements for IoT applications. The efficiency experiments conducted indicate that the BIW module is well-suited to IoT application scenarios. The benchmark models enhanced with BIW can achieve real-time detection performance on real-world data sets.

V. CONCLUSION

In this article, a novel lightweight deepfake frame-level detection result aggregation method called BIW is proposed. To improve the robustness of deepfake frame-level detection models, BIW applies Bayes' theorem to infer the weights of frame detection results. Instead of the naive averaging method, BIW tends to give higher weights to frames with more credible results. Moreover, three deepfake data sets with video

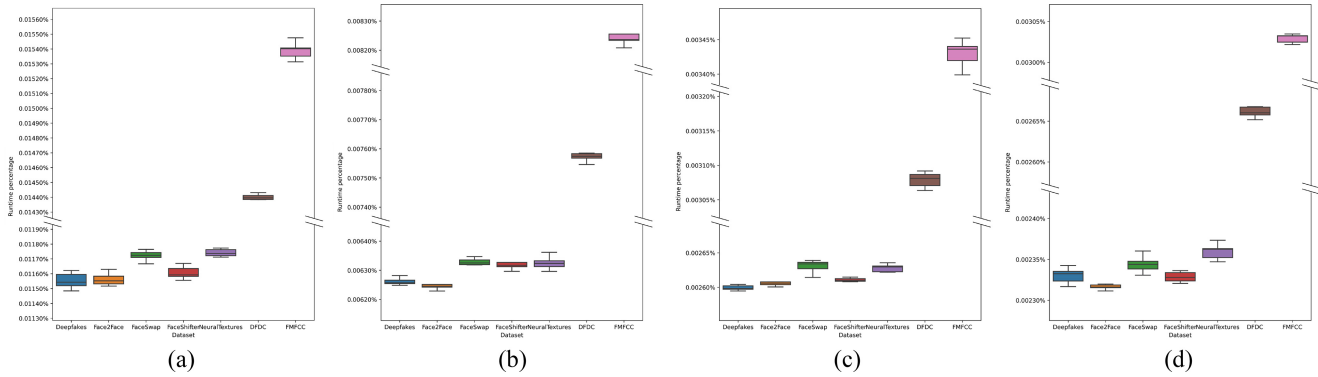


Fig. 6. Proportion of runtime taken by the BIW module when the four baselines detecting videos on five real-world data sets. We repeated the simulated detection process ten times, excluding any anomalous data resulting from the loading of model caches. (a) Meso4. (b) MesoInception4. (c) XceptionNet. (d) EfficientNet-B4.

TABLE IX
AVERAGE PROCESSING TIME OF A ONE-SECOND VIDEO USING THE ENHANCED BIW MODEL (IN MILLISECONDS)

Model \ Dataset	IIE-FACE	FF++					DFDC
		DF	F2F	FS	FST	NT	
Meso4 BIW	5.76	22.86	24.48	23.63	24.45	23.60	3.95
MesoInception4 BIW	10.78	42.18	47.42	45.60	47.42	45.59	9.40
XceptionNet BIW	31.02	148.63	170.41	166.32	170.24	166.11	28.37
EfficientNet-B4 BIW	39.26	167.78	189.58	185.20	189.49	184.68	35.39
Resolution	640 × 360	640 × 480	640 × 480	640 × 480	640 × 480	640 × 480	1920 × 1080 1440 × 2560

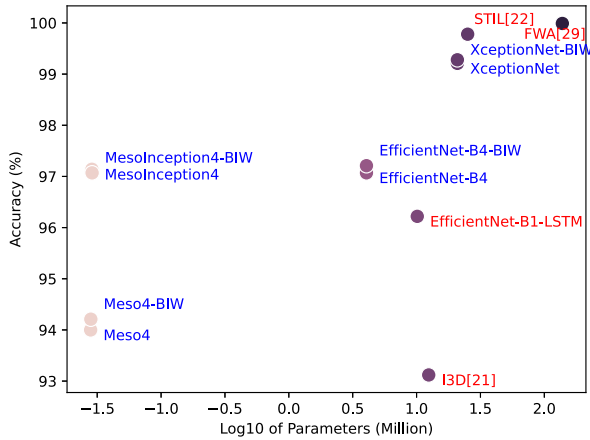


Fig. 7. Performance and resource consumption of frame-level detection models (marked with blue) and video-level detection models (marked with red).

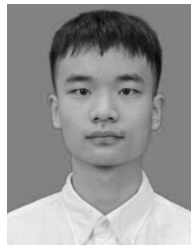
interference are designed to evaluate the robustness of existing deepfake detection models with and without our proposed BIW addition. According to the experimental results on three special real-world deepfake data sets and IoT platforms, BIW has proven to be capable of significantly enhancing the robustness of existing deepfake frame-level detection models and is an effective and resource-free defense for securing IoT systems against deepfake attacks.

REFERENCES

- [1] R. Katarya and A. Lal, "A study on combating emerging threat of deepfake weaponization," in *Proc. 4th Int. Conf. I-SMAC (IoT Social, Mobile, Analytics Cloud) (I-SMAC)*, 2020, pp. 485–490.
- [2] Y. Ding et al., "An efficient industrial federated learning framework for AIoT: A face recognition application," 2022, *arXiv:2206.13398*.

- [3] J. Botha and H. Pieterse, "Fake news and deepfakes: A dangerous threat for 21st century information security," in *Proc. Int. Conf. Cyber Warfare Security*, 2020, pp. 57–66.
- [4] S. Tariq, S. Jeon, and S. S. Woo, "Am I a real or fake celebrity? Evaluating face recognition and verification APIs under deepfake impersonation attack," in *Proc. ACM Web Conf.*, 2022, pp. 512–523.
- [5] Y. Roth and A. Achuthan, "Building rules in public: Our approach to synthetic & manipulated media," Twitter Blog, 2020. [Online]. Available: https://blog.twitter.com/en_us/topics/company/2020/new-approach-to-synthetic-and-manipulated-media
- [6] "Enforcing against manipulated media." Meta. 2020. [Online]. Available: <https://about.fb.com/news/2020/01/enforcing-against-manipulated-media/>
- [7] T. Hsu, "Worries grow that TikTok is new home for manipulated video and photos." New York Times. 2022. [Online]. Available: <https://www.nytimes.com/2022/11/04/technology/tiktok-deepfakes-disinformation.html>
- [8] D. Afchar, V. Nozick, J. Yamagishi, and I. Echizen, "MesoNet: A compact facial video forgery detection network," in *Proc. IEEE Int. Workshop Inf. Forensics Security (WIFS)*, 2018, pp. 1–7.
- [9] A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Nießner, "FaceForensics++: Learning to detect manipulated facial images," in *Proc. Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 1–14.
- [10] Y. Qian, G. Yin, L. Sheng, Z. Chen, and J. Shao, "Thinking in frequency: Face forgery detection by mining frequency-aware clues," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 86–103.
- [11] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6105–6114.
- [12] U. A. Ciftci, I. Demir, and L. Yin, "How do the hearts of deep fakes beat? Deep fake source detection via interpreting residuals with biological signals," in *Proc. IEEE Int. Joint Conf. Biometr. (IJCB)*, 2020, pp. 1–10.
- [13] Y. Li, M.-C. Chang, and S. Lyu, "In Ictu Oculi: Exposing AI created fake videos by detecting eye blinking," in *Proc. IEEE Int. Workshop Inf. Forensics Security (WIFS)*, 2018, pp. 1–7.
- [14] J. A. Redi, W. Taktak, and J.-L. Dugelay, "Digital image forensics: A booklet for beginners," *Multimedia Tools Appl.*, vol. 51, no. 1, pp. 133–162, 2011.
- [15] J. Fridrich and J. Kodovsky, "Rich models for steganalysis of digital images," *IEEE Trans. Inf. Forensics Security*, vol. 7, pp. 868–882, 2012.
- [16] D. Cozzolino, G. Poggi, and L. Verdoliva, "Recasting residual-based local descriptors as convolutional neural networks: An application to image forgery detection," in *Proc. 5th ACM Workshop Inf. Hiding Multimedia Security*, 2017, pp. 159–164.

- [17] W. Li, L. Fan, Z. Wang, C. Ma, and X. Cui, "Tackling mode collapse in multi-generator GANs with orthogonal vectors," *Pattern Recognit.*, vol. 110, Feb. 2021, Art. no. 107646.
- [18] I. Korshunova, W. Shi, J. Dambre, and L. Theis, "Fast face-swap using convolutional neural networks," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 3677–3685.
- [19] I. Amerini, L. Galteri, R. Caldelli, and A. Del Bimbo, "Deepfake video detection through optical flow based CNN," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshops*, 2019, pp. 1–3.
- [20] Z. Li, C. Ma, X. Shi, D. Zhang, W. Li, and L. Wu, "TSA-GAN: A robust generative adversarial networks for time series augmentation," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2021, pp. 1–8.
- [21] W. Li et al., "Sketch-then-edit generative adversarial network," *Knowl.-Based Syst.*, vol. 203, Sep. 2020, Art. no. 106102. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950705120303701>
- [22] J. Carreira and A. Zisserman, "Quo vadis, action recognition? A new model and the kinetics dataset," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 6299–6308.
- [23] Z. Gu et al., "Spatiotemporal inconsistency learning for DeepFake video detection," in *Proc. 29th ACM Int. Conf. Multimedia*, 2021, pp. 3473–3481.
- [24] T. Zhao, X. Xu, M. Xu, H. Ding, Y. Xiong, and W. Xia, "Learning self-consistency for deepfake detection," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 15023–15033.
- [25] W. Pu et al., "Learning a deep dual-level network for robust DeepFake detection," *Pattern Recognit.*, vol. 130, Oct. 2022, Art. no. 108832.
- [26] D. Wodajo and S. Atnafu, "Deepfake video detection using convolutional vision transformer," 2021, *arXiv:2102.11126*.
- [27] H. Zhao, W. Zhou, D. Chen, T. Wei, W. Zhang, and N. Yu, "Multi-attentional deepfake detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 2185–2194.
- [28] T. Wang, H. Cheng, K. P. Chow, and L. Nie, "Deep convolutional pooling transformer for deepfake detection," 2022, *arXiv:2209.05299*.
- [29] M. S. Rana and A. H. Sung, "DeepfakeStack: A deep ensemble-based learning technique for deepfake detection," in *Proc. 7th IEEE Int. Conf. Cyber Security Cloud Comput. (CSCloud)/Proc. 6th IEEE Int. Conf. Edge Comput. Scalable Cloud (EdgeCom)*, 2020, pp. 70–75.
- [30] D. M. Montserrat et al., "Deepfakes detection with automatic face weighting," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, 2020, pp. 668–669.
- [31] X. Li et al., "Sharp multiple instance learning for DeepFake video detection," in *Proc. 28th ACM Int. Conf. Multimedia*, 2020, pp. 1864–1872.
- [32] J. Hu, X. Liao, J. Liang, W. Zhou, and Z. Qin, "FInfer: Frame inference-based deepfake detection for high-visual-quality videos," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, 2022, pp. 951–959.
- [33] B. Dolhansky, R. Howes, B. Pfau, N. Baram, and C. C. Ferrer, "The deepfake detection challenge (DFDC) preview dataset," 2019, *arXiv:1910.08854*.
- [34] "Fake media forensic challenge of CSIG." CSIG. 2022. [Online]. Available: <http://fmfcc.net/contest-introduction>
- [35] A. Bansal, S. Ma, D. Ramanan, and Y. Sheikh, "Recycle-GAN: Unsupervised video retargeting," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 119–135.
- [36] S. Pidhorskyi, D. A. Adjeroh, and G. Doretto, "Adversarial latent autoencoders," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 14104–14113.
- [37] A. Rahman, M. S. Hossain, N. A. Alrajeh, and F. Alsolami, "Adversarial examples—Security threats to COVID-19 deep learning systems in medical IoT devices," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9603–9610, Jun. 2021.
- [38] P. Abichandani, D. Lobo, S. Kabrawala, and W. McIntyre, "Secure communication for multiquadrotor networks using Ethereum blockchain," *IEEE Internet Things J.*, vol. 8, no. 3, pp. 1783–1796, Feb. 2021.
- [39] S. Agarwal, H. Farid, Y. Gu, M. He, K. Nagano, and H. Li, "Protecting world leaders against deep fakes," in *Proc. CVPR Workshops*, vol. 1, 2019, p. 38.
- [40] B. Dolhansky et al., "The deepfake detection challenge (DFDC) dataset," 2020, *arXiv:2006.07397*.
- [41] S. Singh, J. G. Andrews, and G. de Veciana, "Interference shaping for improved quality of experience for real-time video streaming," *IEEE J. Sel. Areas Commun.*, vol. 30, no. 7, pp. 1259–1269, Aug. 2012.
- [42] Z. Ren et al., "AIPerf: Automated machine learning as an AI-HPC benchmark," *Big Data Min. Anal.*, vol. 4, no. 3, pp. 208–220, 2021.



Linjiang Zhou (Graduate Student Member, IEEE) received the B.E. degree from the School of Cyber Science and Engineering, Wuhan University, Wuhan, China, in 2022, where he is currently pursuing the Ph.D. degree.

His main research interests include explainable artificial intelligence, edge computing, deepfake detection, and time series classification.



Chao Ma (Member, IEEE) received the B.S. degree in computer science and the Ph.D. degree in software theory from Wuhan University, Wuhan, China, in 2005 and 2010, respectively.

He is currently an Assistant Professor with the School of Cyber Science and Engineering, Wuhan University. He has published over 50 academic papers in major international journals and conference proceedings. His research interests include time series analytics, generative AI, explainable AI, and reinforcement learning.

Dr. Ma is currently the Professional Member of CCF and the Life-Long Member of CAAI.



Zepeng Wang (Student Member, IEEE) is currently pursuing the Ph.D. degree in cyberspace security with the School of Cyber Science and Engineering, Wuhan University, Wuhan, China.

His research interests include safe reinforcement learning and robust reinforcement learning.



Yixuan Zhang (Student Member, IEEE) is currently pursuing the B.E. degree in information and engineering with Chongqing University of Posts and Telecommunications, Chongqing, China.

He has some experience in software design.



Xiaochuan Shi (Member, IEEE) received the B.S. degree in computer science and technology, the M.S. degree in software theory, and the Ph.D. degree in computer architecture from Wuhan University, Wuhan, China, in 2006, 2008, and 2011, respectively.

From 2009 to 2010, he was a Research Assistant with the P.C. Rossin College of Engineering and Applied Science, Lehigh University, Bethlehem, PA, USA. From 2012 to 2014, he was a Postdoctoral Fellow and a Visiting Scholar with the Department of Computer Science and Engineering, The Pennsylvania State University, Pennsylvania, PA, USA. He is currently an Associate Professor with the School of Cyber Science and Engineering, Wuhan University. He has published more than 50 academic papers, covering the fields of big data analysis, reinforcement learning, deep learning, and wireless sensor networks.



Libing Wu (Member, IEEE) received the Ph.D. degree from Wuhan University, Wuhan, China, in 2006.

He is currently a Professor with the School of Cyber Science and Engineering, Wuhan University. He was a Visiting Scholar with the Advanced Networking Lab, University of Kentucky, Lexington, KY, USA, in 2011. He was a Senior Visiting Fellow with the State University of New York, Buffalo, NY, USA, in 2017. His research interests include network security, Internet of Things, machine learning, and data security.